Document Conversion Service

Complete User Guide

Version 2.0 • 2025

A production-grade microservice that converts documents and images to plain text.
Supports SQS, RabbitMQ, Kafka, Temporal.io durable workflows, REST API, and 10 document formats.

Table of Contents

1. Overview
2. Architecture
3. Prerequisites
4. Quick Start – Docker Compose
5. Configuration Reference
6. Message Schema
7. REST API Reference
8. Using the API Test Console
9. Message Bus Integration
   9.1 Amazon SQS
   9.2 RabbitMQ
   9.3 Apache Kafka
10. Temporal.io Durable Workflows
   10.1 Overview & Architecture
   10.2 The DocumentConversionWorkflow
   10.3 Document-Type Child Workflows
   10.4 Activities
   10.5 Batch Processing
   10.6 The Temporal Worker
   10.7 Temporal Web UI
   10.8 Workflow API Endpoints
   10.9 Execution Modes
   10.10 Starting Workflows from Python
11. Document Converters
12. Image OCR
13. File Fetchers (S3, URL, FTP)
14. Output Storage (S3)
15. Memory Efficiency & Streaming
16. Testing
17. Troubleshooting
18. curl Examples Cookbook

1. Overview

The Document Conversion Service is a containerized Python microservice that accepts conversion jobs from multiple sources – three message buses (Amazon SQS, RabbitMQ, Apache Kafka), Temporal.io durable workflows, and an optional REST API – and converts documents and images into plain text. The extracted text is uploaded to an Amazon S3 bucket (or S3-compatible storage like LocalStack or MinIO).

Every component is independently toggleable via environment variables. You can run the service as a pure queue consumer, a Temporal workflow worker, an API-only service, or any combination.

Key Features

- 10 document formats: PDF, DOCX, XLSX, PPTX, HTML, RTF, ODT, TXT, CSV, Image (OCR)
- 3 message buses: SQS, RabbitMQ, Kafka – all using the same JSON message schema
- Temporal.io durable execution: automatic retries, crash recovery, queryable state, full audit trail
- Per-document-type Temporal child workflows with specialized orchestration
- Batch workflow: convert multiple documents in parallel via a single Temporal invocation
- REST API with file upload, JSON job submission, and workflow status endpoints
- Web-based API Test Console (built-in HTML UI) + Temporal Web UI on port 8088
- 3 document sources: S3, HTTP/HTTPS URLs, FTP servers
- Authentication support: None, Basic, Bearer token
- Memory-efficient streaming via Python generators and temporary files
- PDF image extraction with OCR (Tesseract)
- Full Docker Compose stack with LocalStack, RabbitMQ, Kafka, Temporal, FTP
- Every component can be enabled/disabled via environment variables
- Automatic fallback from Temporal to direct processing if Temporal is unavailable

2. Architecture

System Diagram

The service follows a pipeline pattern: Ingest → Fetch → Convert → Upload. When Temporal is enabled, each step becomes a durable, retryable activity within a workflow.

```
┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐
┌──────────────┐
│    SQS     │ │ RabbitMQ  │ │   Kafka   │ │ REST API  │ │  Temporal.io   │
│  Queue     │ │  Queue    │ │  Topic    │ │ (FastAPI) │ │  Client SDK    │
└──────┬──────┘ └──────┬──────┘ └──────┬──────┘ └──────┬──────┘
└──────┬──────┘
      │            │            │            │             │
      └────────────┴────────┬───┴────────────┘             │
                            ▼                              ▼

┌──────────────────────────────────────────────────────────────────────┐
            │              Processor (routing layer)                   │
            │  USE_TEMPORAL_WORKFLOWS=true → Temporal Workflow          │
            │  USE_TEMPORAL_WORKFLOWS=false → Direct Pipeline           │

└──────────────────────────────┬──────────────────────────────────────┘
                               ▼
           ┌──────────────────────────────────────────────────────────┐
           │        DocumentConversionWorkflow (Temporal)             │
           │  Activity 1: Fetch   (S3 / URL / FTP → tmp file)         │
           │  Child WF:   Convert (PDF / DOCX / XLSX / ... )          │
           │  Activity 3: Upload  (tmp → S3 output bucket)            │
           │  Activity 4: Cleanup (remove tmp files)                  │
           └──────────────────────────────────────────────────────────┘
```

Docker Services

Service	Image	Ports	Purpose
localstack	localstack/localstack:3.4	4566	SQS queues + S3 buckets
rabbitmq	rabbitmq:3.13-management-alpine	5672, 15672	RabbitMQ bus + Management UI
zookeeper	confluentinc/cp-zookeeper:7.6.1	2181	Kafka coordination
kafka	confluentinc/cp-kafka:7.6.1	9092	Kafka message bus
temporal-postgresql	postgres:16-alpine	—	Temporal persistence DB
temporal-server	temporalio/auto-setup:1.24.2	7233	Temporal gRPC frontend
temporal-admin-tools	temporalio/admin-tools:1.24.2	—	Temporal CLI tools
temporal-ui	temporalio/ui:2.26.2	8088	Temporal Web UI

ftp	fauria/vsftpd	21, 21100-21110	Demo FTP server
docconv-app	Built from Dockerfile	8080	The conversion service

Project Structure

```
docconv-service/
├── docker-compose.yml          # Full stack (10 services)
├── Dockerfile
├── requirements.txt
├── .env.example
├── config/settings.py          # Pydantic settings
├── app/
│   ├── main.py                 # ★ Entry point
│   ├── models.py               # Pydantic models
│   ├── api.py                  # FastAPI + workflow endpoints
│   ├── processor.py            # Direct / Temporal routing
│   ├── storage.py              # S3 output uploader
│   ├── static/index.html       # API Test Console
│   ├── bus/                    # SQS, RabbitMQ, Kafka listeners
│   ├── fetchers/              # S3, URL, FTP download
│   ├── converters/            # One module per document type
│   └── workflows/             # ★ Temporal.io integration
│       ├── dataclasses.py     #   Serializable I/O types
│       ├── activities.py      #   12 Temporal activities
│       ├── document_workflows.py # 9 child workflows
│       ├── conversion_workflow.py # Main + Batch workflow
│       ├── worker.py          #   Worker thread
│       └── client.py          #   Start workflows from code
├── scripts/
│   ├── init-localstack.sh
│   └── demo.py
└── tests/
    └── test_full_flow.py      # 48 integration tests
```

3. Prerequisites

Requirement	Version	Notes
Docker	20.10+	Docker Desktop or Docker Engine
Docker Compose	v2.0+	Included with Docker Desktop
Disk space	~4 GB	For container images (including Temporal)
RAM	8 GB recommended	Kafka + Temporal + LocalStack need headroom
Python	3.10+ (optional)	Only for running demo.py or tests locally

Note: All service dependencies are installed inside the Docker image.

4. Quick Start – Docker Compose

Step 1: Extract and enter the project

```
unzip docconv-service.zip
cd docconv-service
```

Step 2: Make the init script executable

```
chmod +x scripts/init-localstack.sh
```

Step 3: Build and start all services

```
docker compose up --build -d
```

Builds the app, pulls all infrastructure images (LocalStack, RabbitMQ, Kafka, Temporal), and starts 10 containers.

Step 4: Verify services are healthy

```
docker compose ps
```

All services should show 'healthy' or 'running'. Temporal may take 30-40 seconds.

Step 5: Open the API Test Console

Open http://localhost:8080 in your browser.

Step 6: Open the Temporal Web UI

Open http://localhost:8088 to view workflows, activities, and execution history.

Step 7: Watch logs

```
docker compose logs -f docconv-app
```

Step 8: Run the demo script (optional)

```
pip install boto3 pika confluent-kafka requests
python scripts/demo.py
```

Step 9: Stop the stack

```
docker compose down          # Stop
docker compose down -v       # Stop + remove volumes
```

5. Configuration Reference

All configuration is via environment variables. Copy .env.example to .env for local overrides.

Feature Flags

Variable	Default	Description
ENABLE_API	true	Start the FastAPI HTTP server
ENABLE_SQS	true	Start the SQS listener thread
ENABLE_RABBITMQ	true	Start the RabbitMQ consumer thread
ENABLE_KAFKA	true	Start the Kafka consumer thread
ENABLE_TEMPORAL	true	Start the Temporal worker thread
USE_TEMPORAL_WORKFLOWS	true	Route all jobs through Temporal (vs direct pipeline)

Set any flag to "false" to disable that component. When USE_TEMPORAL_WORKFLOWS is false, jobs still process successfully via the direct pipeline even if ENABLE_TEMPORAL is true (the worker runs but isn't invoked).

SQS Settings

Variable	Default	Description
SQS_ENDPOINT_URL	http://localstack:4566	SQS endpoint
SQS_QUEUE_NAME	docconv-jobs	Queue to poll
SQS_POLL_INTERVAL	5	Long-poll wait (seconds)

S3 Output Settings

Variable	Default	Description
S3_ENDPOINT_URL	http://localstack:4566	S3 endpoint for uploads
S3_OUTPUT_BUCKET	docconv-output	Default output bucket

RabbitMQ Settings

Variable	Default	Description
RABBITMQ_HOST	rabbitmq	Server hostname
RABBITMQ_PORT	5672	AMQP port
RABBITMQ_USER	docconv	Username
RABBITMQ_PASS	docconv	Password
RABBITMQ_QUEUE	docconv-jobs	Queue name

Kafka Settings

Variable	Default	Description
KAFKA_BOOTSTRAP_SERVERS	kafka:29092	Broker address
KAFKA_TOPIC	docconv-jobs	Topic to subscribe
KAFKA_GROUP_ID	docconv-group	Consumer group ID

Temporal.io Settings

Variable	Default	Description
TEMPORAL_HOST	temporal-server:7233	Temporal gRPC endpoint
TEMPORAL_NAMESPACE	default	Temporal namespace
TEMPORAL_TASK_QUEUE	docconv-tasks	Worker task queue name
TEMPORAL_WORKFLOW_TIMEOUT	3600	Max workflow duration (seconds)
TEMPORAL_ACTIVITY_TIMEOUT	600	Max single activity duration (seconds)
TEMPORAL_RETRY_MAX_ATTEMPTS	3	Retry attempts per activity on failure

AWS Credentials

Variable	Default	Description
AWS_ACCESS_KEY_ID	test	Used by boto3
AWS_SECRET_ACCESS_KEY	test	Used by boto3
AWS_DEFAULT_REGION	us-east-1	AWS region

Tuning

Variable	Default	Description
CHUNK_SIZE	65536	Read/write chunk size (64 KB)
TMP_DIR	/tmp/docconv	Temporary file directory
LOG_LEVEL	INFO	Logging level
API_PORT	8080	HTTP server port

6. Message Schema

Every bus, the REST API, and Temporal workflows accept the same JSON schema.

Full JSON Schema

```
{
  "job_id": "optional-caller-id",
  "document_type": "pdf",
  "location_type": "s3",
  "s3_bucket": "my-bucket",
  "s3_key": "path/to/file.pdf",
  "s3_endpoint_url": null,
  "url": "https://example.com/doc.pdf",
  "ftp_host": "ftp.example.com",
  "ftp_port": 21,
  "ftp_path": "/path/to/file",
  "ftp_user": "user",  "ftp_pass": "pass",
  "auth_type": "none",
  "auth_username": null,  "auth_password": null,
  "auth_token": null,
  "output_s3_bucket": null,
  "output_s3_key": null
}
```

Supported document_type values

Value	Description	Converter / Child Workflow
pdf	PDF with embedded image OCR	pdf_converter / PDFConversionWorkflow
docx	Microsoft Word	docx_converter / DOCXConversionWorkflow
xlsx	Microsoft Excel	xlsx_converter / XLSXConversionWorkflow
pptx	Microsoft PowerPoint	pptx_converter / PPTXConversionWorkflow
html	HTML web pages	html_converter / HTMLConversionWorkflow
rtf	Rich Text Format	rtf_converter / RTFConversionWorkflow
odt	OpenDocument Text	odt_converter / ODTConversionWorkflow
txt	Plain text (encoding detection)	text_converter / TXTConversionWorkflow
csv	Comma-separated values	xlsx_converter / XLSXConversionWorkflow
image	JPEG/PNG/TIFF/BMP/WEBP (OCR)	image_converter / ImageConversionWorkflow

7. REST API Reference

Base URL: http://localhost:8080

GET /

Returns the API Test Console HTML page.

GET /health

Returns service status and all enabled components.

```
curl http://localhost:8080/health
```

Response:

```
{"status":"ok","api_enabled":true,"sqs_enabled":true,
 "rabbitmq_enabled":true,"kafka_enabled":true,
 "temporal_enabled":true,"temporal_workflows_active":true,
 "temporal_host":"temporal-server:7233",
 "temporal_task_queue":"docconv-tasks"}
```

POST /convert/job

Submit a remote-source conversion job.

```
curl -X POST http://localhost:8080/convert/job \
  -H "Content-Type: application/json" \
  -d '{"document_type":"html","location_type":"url",
       "url":"https://example.com"}'
```

Response:

```
{"job_id":"a1b2c3d4e5f6","success":true,
 "output_bucket":"docconv-output",
 "output_key":"converted/a1b2c3d4e5f6.txt",
 "characters_extracted":1256}
```

POST /convert/upload

Upload a file directly (multipart form).

```
curl -X POST http://localhost:8080/convert/upload \
  -F "file=@report.pdf" -F "document_type=pdf"
```

GET /workflow/{id}/status

Query status of a Temporal workflow.

```
curl http://localhost:8080/workflow/docconv-a1b2c3d4e5f6/status
```

Response:

```
{"workflow_id":"docconv-a1b2c3d4e5f6",
 "status":"COMPLETED","custom_status":"COMPLETED",
 "current_step":"DONE","task_queue":"docconv-tasks"}
```

POST /workflow/{id}/cancel

Send a cancellation signal to a running workflow.

```
curl -X POST http://localhost:8080/workflow/docconv-abc123/cancel
```

Response:

```
{"workflow_id":"docconv-abc123",
 "message":"Cancellation signal sent"}
```

GET /workflows/recent

List recent workflows from Temporal.

```
curl http://localhost:8080/workflows/recent?limit=5
```

GET /docs

FastAPI auto-generated Swagger/OpenAPI docs.

8. Using the API Test Console

Open http://localhost:8080 for the built-in web UI.

- Health Check tab – verify connectivity and see all enabled components (including Temporal status)
- Upload tab – drag-and-drop files with auto-detection of document type
- Job tab – build JSON jobs for S3/URL/FTP sources with live preview
- Schema tab – full message reference with supported values
- Request history – logs every request with method, status code, and timing

9. Message Bus Integration

Each listener runs in its own daemon thread. When USE_TEMPORAL_WORKFLOWS is enabled, messages received from any bus are routed through a Temporal workflow for durable execution.

9.1 Amazon SQS

Long-polls the configured SQS queue. Uses LocalStack locally; point to real AWS in production.

Sending via Python

```
import boto3, json
sqs = boto3.client("sqs", endpoint_url="http://localhost:4566",
    aws_access_key_id="test", aws_secret_access_key="test", region_name="us-east-1")
url = sqs.get_queue_url(QueueName="docconv-jobs")["QueueUrl"]
sqs.send_message(QueueUrl=url, MessageBody=json.dumps(
    {"document_type":"pdf","location_type":"s3",
     "s3_bucket":"docconv-input","s3_key":"test.pdf"}))
```

9.2 RabbitMQ

Connects via AMQP. Management UI at http://localhost:15672 (docconv/docconv).

Sending via Python

```
import pika, json
creds = pika.PlainCredentials("docconv","docconv")
conn =
pika.BlockingConnection(pika.ConnectionParameters("localhost",5672,credentials=creds))
ch = conn.channel(); ch.queue_declare(queue="docconv-jobs",durable=True)
ch.basic_publish(exchange="",routing_key="docconv-jobs",
    body=json.dumps({"document_type":"html","location_type":"url",
                     "url":"https://example.com"}),
    properties=pika.BasicProperties(delivery_mode=2))
conn.close()
```

9.3 Apache Kafka

Subscribes to the configured topic. Auto-topic creation is enabled.

Sending via Python

```
from confluent_kafka import Producer
import json
```

```
p = Producer({"bootstrap.servers":"localhost:9092"})
p.produce("docconv-jobs",value=json.dumps(
    {"document_type":"docx","location_type":"url",
     "url":"https://example.com/doc.docx"}).encode("utf-8"))
p.flush()
```

10. Temporal.io Durable Workflows

10.1 Overview & Architecture

Temporal.io is an open-source durable execution system that guarantees workflows run to completion even if the worker process crashes, restarts, or the server reboots. When enabled, every document conversion job is executed as a Temporal workflow, giving you automatic retries, queryable state, cancellation signals, a full audit trail, and crash recovery – for free.

The integration uses a three-tier design: a top-level DocumentConversionWorkflow orchestrates the pipeline, delegates conversion to one of 9 document-type-specific child workflows, and each child workflow invokes a dedicated Temporal activity. Every step is independently retryable with exponential backoff.

```
DocumentConversionWorkflow (top-level)
  │
  ├─ Activity: fetch_document     (S3 / URL / FTP → tmp file)
  │
  ├─ Child WF: PDFConversionWorkflow
  │     └─ Activity: convert_pdf   (pdfplumber + OCR → text)
  │   OR
  ├─ Child WF: DOCXConversionWorkflow
  │     └─ Activity: convert_docx  (python-docx → text)
  │   OR ... (one child WF per document type)
  │
  ├─ Activity: upload_text        (text → S3 output bucket)
  │
  └─ Activity: cleanup_temp_files (remove tmp files)
```

10.2 The DocumentConversionWorkflow

The main workflow (conversion_workflow.py) orchestrates 4 sequential steps: Fetch, Convert, Upload, Cleanup. It provides three Temporal features that make it observable and controllable:

- **Queryable state:** Queryable state:
- Use the "get_status" query to check the current phase (FETCHING, CONVERTING_PDF, UPLOADING, COMPLETED, FAILED). Use "get_step" for the specific sub-step.
- **Cancellation signal:** Cancellation signal:
- Send the "cancel" signal to gracefully stop a running workflow. The workflow checks for cancellation between each step and returns a CANCELLED result.
- **Automatic fallback:** Automatic fallback:
- If a Temporal workflow fails to start (e.g. server unreachable), the processor automatically falls back to the direct (inline) pipeline so jobs never get stuck.

Workflow ID format: docconv-{job_id} (e.g. docconv-a1b2c3d4e5f6)

10.3 Document-Type Child Workflows

Each document type has a dedicated child workflow that wraps a single conversion activity. This gives per-type visibility in the Temporal UI and allows type-specific retry policies or timeout tuning.

Document Type	Child Workflow	Activity Name
PDF	PDFConversionWorkflow	convert_pdf
DOCX	DOCXConversionWorkflow	convert_docx
XLSX / CSV	XLSXConversionWorkflow	convert_xlsx
PPTX	PPTXConversionWorkflow	convert_pptx
HTML	HTMLConversionWorkflow	convert_html
RTF	RTFConversionWorkflow	convert_rtf
ODT	ODTConversionWorkflow	convert_odt
TXT	TXTConversionWorkflow	convert_txt
Image (OCR)	ImageConversionWorkflow	convert_image

The child workflow ID format is: docconv-convert-{job_id}

10.4 Activities

Activities are the individual, retryable steps executed by the Temporal worker. There are 12 activities registered:

Activity	Module	Description
fetch_document	activities.py	Download source file to local tmp (S3/URL/FTP)
convert_pdf	activities.py	PDF text extraction + embedded image OCR
convert_docx	activities.py	DOCX paragraph + table extraction
convert_xlsx	activities.py	XLSX/CSV spreadsheet extraction
convert_pptx	activities.py	PPTX slide text + speaker notes
convert_html	activities.py	HTML tag stripping
convert_rtf	activities.py	RTF control word stripping
convert_odt	activities.py	OpenDocument text extraction
convert_txt	activities.py	Plain text with encoding detection
convert_image	activities.py	Image OCR via Tesseract
upload_text	activities.py	Upload converted text to S3
cleanup_temp_files	activities.py	Remove temporary files

Every activity sends heartbeats during long operations (every 10 chunks for converters, at key checkpoints for fetch/upload). The heartbeat timeout is 120 seconds – if the Temporal server doesn't receive a heartbeat within that window, it considers the worker dead and reschedules the activity on another worker.

Retry policy: exponential backoff starting at 5 seconds, doubling up to 60 seconds, with a maximum of TEMPORAL_RETRY_MAX_ATTEMPTS (default 3) attempts per activity.

10.5 Batch Processing

The BatchConversionWorkflow accepts a list of ConversionWorkflowInput objects and launches them as parallel child workflows. Each child runs independently with its own retry logic. The batch workflow provides a queryable progress counter.

```
# Start a batch workflow
handle = await client.start_workflow(
    "BatchConversionWorkflow",
    [input1, input2, input3],
    id="batch-001",
    task_queue="docconv-tasks")

# Query progress
progress = await handle.query("get_batch_progress")
# {"total": 3, "completed": 2, "failed": 0}
```

10.6 The Temporal Worker

The worker (worker.py) runs in a daemon thread alongside the API and bus listeners. It registers all 11+ workflows and 12 activities, connects to the Temporal server with automatic retry (30 attempts, 3-second intervals), and polls the docconv-tasks queue for work.

To run the worker separately (e.g. on a dedicated host), set ENABLE_API=false, ENABLE_SQS=false, ENABLE_RABBITMQ=false, ENABLE_KAFKA=false, ENABLE_TEMPORAL=true.

10.7 Temporal Web UI

The Temporal Web UI is available at http://localhost:8088 and provides:

- Workflow list with filtering by status, type, and ID
- Detailed execution history for each workflow showing every activity attempt
- Visual timeline of workflow execution with durations
- Query panel to run get_status and get_step queries on running workflows
- Signal panel to send cancel signals to running workflows
- Stack trace view for debugging failed activities

10.8 Workflow API Endpoints

Three REST endpoints interact with Temporal workflows:

```
GET  /workflow/{workflow_id}/status  - Query current state
POST /workflow/{workflow_id}/cancel  - Send cancellation signal
GET  /workflows/recent?limit=20     - List recent workflows
```

The workflow_id follows the format docconv-{job_id}. For example, if you submit a job with job_id='abc123', query status at /workflow/docconv-abc123/status.

10.9 Execution Modes

The service supports two execution modes, controlled by environment variables:

Setting	Behavior
USE_TEMPORAL_WORKFLOWS=true	All jobs route through Temporal. Full durability, retries, and visibility.
USE_TEMPORAL_WORKFLOWS=false	Jobs process inline (direct pipeline). Faster for dev/testing.
ENABLE_TEMPORAL=false	Temporal worker doesn't start. All jobs use direct pipeline regardless of USE_TEMPORAL_WORKFLOWS.
Temporal server down	Automatic fallback to direct pipeline with a warning log.

10.10 Starting Workflows from Python

You can start workflows directly from Python code using the client helper:

```
from app.workflows.client import start_conversion_workflow_sync
from app.models import ConversionJob

job = ConversionJob(
    job_id="my-job-001",
    document_type="pdf",
    location_type="s3",
    s3_bucket="docconv-input",
    s3_key="docs/report.pdf",
)

result = start_conversion_workflow_sync(job)
print(f"Success: {result.success}")
print(f"Output: s3://{result.output_bucket}/{result.output_key}")
```

For async code, use start_conversion_workflow() instead.

11. Document Converters

Each converter is a standalone Python module yielding text chunks via generators.

PDF (pdf_converter.py)

Extracts selectable text via pdfplumber page-by-page. Falls back to full-page OCR for scanned pages. Extracts and OCRs embedded images tagged as [IMAGE TEXT].

DOCX (docx_converter.py)

Reads paragraphs in batches of 50 using python-docx. Extracts table content with cell delimiters.

XLSX / CSV (xlsx_converter.py)

XLSX: reads each sheet with openpyxl in read-only mode, 200 rows per batch. CSV: stdlib csv module.

PPTX (pptx_converter.py)

Reads slide shapes (text frames + tables) and speaker notes. One slide per yield.

HTML (html_converter.py)

Strips script/style tags via BeautifulSoup + lxml. Chunks by size.

RTF (rtf_converter.py)

Strips RTF control words via striprtf. Chunks by size.

ODT (odt_converter.py)

Reads OpenDocument paragraphs via odfpy. 50 paragraphs per batch.

TXT (text_converter.py)

Detects encoding via chardet, reads in CHUNK_SIZE chunks.

12. Image OCR

Handles JPEG, PNG, TIFF (multi-frame), BMP, WEBP via Pillow + Tesseract. Down-scales images >4000px. Converts to RGB if needed. PDF converter also extracts and OCRs embedded images.

13. File Fetchers (S3, URL, FTP)

Each fetcher streams to a local tmp file in CHUNK_SIZE chunks using generators.

S3 Fetcher

- Uses boto3 streaming
- Supports custom endpoint URLs for LocalStack/MinIO

URL Fetcher

- requests with stream=True
- Supports none/basic/bearer auth
- 120s timeout

FTP Fetcher

- stdlib ftplib retrbinary
- Supports anonymous and authenticated access

14. Output Storage (S3)

Text chunks write to a local tmp file, then upload to S3. Default path:

```
s3://{S3_OUTPUT_BUCKET}/converted/{job_id}.txt
```

Checking Output

```
aws --endpoint-url=http://localhost:4566 s3 ls s3://docconv-output/converted/
--recursive
aws --endpoint-url=http://localhost:4566 s3 cp
s3://docconv-output/converted/JOB_ID.txt -
```

15. Memory Efficiency & Streaming

- **Fetchers:** Stream to disk in CHUNK_SIZE chunks via generators. Auto-deleted after processing.
- **Converters:** Yield text one page/chunk at a time. Never buffer full text in memory.
- **Upload:** Writes chunks to a tmp file, then uploads to S3. No full-text buffering.
- **PDF images:** Rendered to tmp PNG files for OCR, deleted immediately after each page.
- **API uploads:** Streamed to disk in chunks from the HTTP request body.

- **Temporal:** Activities use the same generators. Files persist across activity boundaries via durable tmp paths.

16. Testing

Running the Integration Test Suite

```
cd docconv-service
pip install -r requirements.txt reportlab httpx temporalio
python -m tests.test_full_flow
```

What the Tests Cover (48 tests)

Section	Count	Description
Sample Generation	10	Creates sample files for every format
Converter Unit Tests	10	Tests each converter module individually
Dispatcher Tests	10	Verifies DocumentType → converter routing
Pipeline Tests	10	End-to-end: file → convert → output
Model Validation	4	Pydantic schema validation and rejection
REST API Tests	7	Health, upload (5 formats), validation
Temporal Tests	7	Dataclasses, input/output mapping, registries, fallback

17. Troubleshooting

Services fail to start

Run "docker compose logs <service>". Ensure ports 4566, 5672, 7233, 8080, 8088, 9092 are free. Increase Docker memory to 8 GB.

Temporal server takes long to start

The temporal-server container runs auto-setup which creates the database schema on first boot. This can take 30-40 seconds. The healthcheck has a 40-second start_period for this reason.

Temporal worker says 'Could not connect'

The worker retries 30 times with 3-second intervals. Ensure temporal-server is healthy: docker compose logs temporal-server. The Temporal gRPC port is 7233.

Workflow shows FAILED in Temporal UI

Click the workflow ID to see the full execution history. Each activity shows its input, output, and error stack trace. Check which activity failed and its retry count.

SQS listener says 'queue not found'

LocalStack may still be initializing. Check that init-localstack.sh is executable.

OCR returns empty text

Ensure tesseract-ocr is installed in the Docker image (it is by default). Check image quality.

PDF conversion is slow

Full-page OCR renders at 200 DPI per page. Selectable-text PDFs are fast (no OCR).

Upload fails with 'Connection refused'

The S3 upload targets LocalStack. Check: curl http://localhost:4566/_localstack/health

RabbitMQ connection refused

Auto-retries every 5 seconds. RabbitMQ may take 10-15 seconds to start.

Kafka consumer not receiving messages

Verify auto-topic creation is enabled. Check bootstrap server address.

API Test Console shows 'Disconnected'

Click 'Test Connection'. If Docker runs in a VM, use the VM's IP instead of localhost.

Temporal fallback keeps triggering

Check TEMPORAL_HOST points to the correct address. Inside Docker it should be temporal-server:7233. From the host it should be localhost:7233.

18. curl Examples Cookbook

Health check

```
curl http://localhost:8080/health
```

Upload a PDF

```
curl -X POST http://localhost:8080/convert/upload \
  -F "file=@document.pdf" -F "document_type=pdf"
```

Upload an image for OCR

```
curl -X POST http://localhost:8080/convert/upload \
  -F "file=@scan.png" -F "document_type=image"
```

Upload DOCX with custom output key

```
curl -X POST http://localhost:8080/convert/upload \
  -F "file=@report.docx" -F "document_type=docx" \
  -F "output_s3_key=reports/2025/q1.txt"
```

Convert from URL

```
curl -X POST http://localhost:8080/convert/job \
  -H "Content-Type: application/json" \
  -d '{"document_type":"html","location_type":"url","url":"https://example.com"}'
```

Convert from URL with Basic auth

```
curl -X POST http://localhost:8080/convert/job \
  -H "Content-Type: application/json" \
  -d '{"document_type":"pdf","location_type":"url",
       "url":"https://secure.example.com/doc.pdf",
       "auth_type":"basic","auth_username":"user","auth_password":"pass"}'
```

Convert from S3 (LocalStack)

```
curl -X POST http://localhost:8080/convert/job \
  -H "Content-Type: application/json" \
  -d '{"document_type":"pdf","location_type":"s3",
       "s3_bucket":"docconv-input","s3_key":"docs/report.pdf",
       "s3_endpoint_url":"http://localstack:4566"}'
```

Convert from FTP

```
curl -X POST http://localhost:8080/convert/job \
  -H "Content-Type: application/json" \
  -d '{"document_type":"txt","location_type":"ftp",
       "ftp_host":"ftp","ftp_path":"/data/file.txt",
       "ftp_user":"docconv","ftp_pass":"docconv"}'
```

Query Temporal workflow status

```
curl http://localhost:8080/workflow/docconv-abc123/status
```

Cancel a running Temporal workflow

```
curl -X POST http://localhost:8080/workflow/docconv-abc123/cancel
```

List recent Temporal workflows

```
curl http://localhost:8080/workflows/recent?limit=10
```

List converted output files

```
aws --endpoint-url=http://localhost:4566 s3 ls \
  s3://docconv-output/converted/ --recursive
```

Download and view converted text

```
aws --endpoint-url=http://localhost:4566 s3 cp \
  s3://docconv-output/converted/JOB_ID.txt -
```